

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра "ПОВТ и АС"

Алгоритмы поиска образца в тексте

Практикум

по выполнению лабораторной работы № 10

по дисциплине "Структуры и алгоритмы обработки данных"

Ростов-на-Дону

20 г.

Составитель: к.ф.-м.н., доц. Габрельян Б.В.

Практикум по выполнению лабораторной работы по дисциплине "Структуры и алгоритмы обработки данных". ДГТУ, г. Ростов на-Дону, 20 г.

В практикуме кратко изложены теоретические вопросы, необходимые для успешного выполнения лабораторной работы, рабочее задание и контрольные вопросы для самопроверки.
Предназначено для обучающихся по направлениям 090304 "Программная инженерия", 020303 "Математическое обеспечение и администрирование информационных систем".

Ответственный за выпуск:

зав. кафедрой Долгов В.В.

Издательский центр ДГТУ, 20

Оглавление

Введение 4

Теоретическая часть 4

1. Базовые концепции 4

2. Линейный поиск 5

3. Алгоритм Бойера-Мура-Хорспулла 6

4. Алгоритм Карпа-Рабина 10

Инструкция по технике безопасности при проведении работы 13

Цель работы 13

Рабочее задание 14

Общие положения 16

Материально-техническое обеспечение работы 16

Порядок выполнения работы 16

Контрольные вопросы 16

Перечень использованных информационных ресурсов 19

Введение

Одной из основных, базовых операций в задачах обработки текста является поиск подстроки в строке или, другими словами, поиск образца в тексте. Результатом поиска должна быть позиция, начиная с которой образец встретился в тексте или некоторое специальное значение, показывающее, что данный образец в тексте отсутствует. В общем случае образец может встречаться в тексте многократно и тогда важной оказывается задача нахождения всех его вхождений. В принципе эту более общую задачу можно свести к многократному повторению более простой операции – поиску первого вхождения образца в тексте.

Существует много алгоритмов точного нахождения образца в тексте, построенных на разных принципах, то есть на основе разных методов создания алгоритмов.

Теоретическая часть

1. Базовые концепции

Текст и образец состоят из символов, принадлежащих предопределенному конечному набору символов. Этот набор символов называют алфавитом – A. Используемый алфавит зависит от задачи. В общем случае, для произвольных тестов, ориентированных на людей это могут быть стандартные наборы: ASCII, Unicode 16, Unicode 32 и т.п. Но в конкретных задачах тексты и образцы могут содержать, например, только символы кириллицы или латиницы. В случае какой-нибудь шифрограммы алфавит может состоять только из десяти арабских цифр и т.д. Таким образом, количество символов в алфавите (мощность алфавита) |A| определяется решаемой задачей.

Задачу поиска образца в тексте можно определить следующим образом. Заданы текст T – последовательность из n символов (символы могут повторяться произвольное количество раз) t_i из алфавита A ($t_i \in A$, $i \in [0 .. n-1]$) и образец P – последовательность из m символов (символы могут повторяться произвольное количество раз) p_i из алфавита A ($p_i \in A$, $i \in [0 .. m-1]$), причем $m \leq n$. Необходимо найти позицию pos в тексте T, такую, что m символов T начиная с позиции pos совпадают с соответствующими символами образца P ($\forall t_{pos+i} \in T$, $i \in [0 .. m-1]$, $t_{pos+i} = p_i$).

2. Линейный поиск

Алгоритм линейного (простого) поиска построен методом "грубой силы". Основная операция – сравнение двух символов, текста и образца, на равенство (или неравенство). Начинаем с начала текста, позиция pos = 0. Просматриваем символы образца, начиная с первого по счету (с индексом i = 0). Сравниваем на равенство P[i] с T[pos + i].

T	**H**	e	l	L	o		W	o	r	l	d
P	**l**	o									
pos	**0**	1	2	3	4	5	6	7	8	9	10

Если не совпадают, сдвигаем образец по тексту на одну позицию вправо. То есть увеличиваем значение pos на единицу, pos = pos + 1.

T	H	**e**	l	l	o		W	o	r	l	d
P		**l**	o								
pos	0	**1**	2	3	4	5	6	7	8	9	10

Снова pos = pos + 1.

T	H	e	**l**	l	o		W	o	r	l	d
P			**l**	o							
pos	0	1	**2**	3	4	5	6	7	8	9	10

Если P[i] = T[pos + i] – переходим к следующим символам текста и образца, то есть увеличиваем на 1 значение i, i = i + 1.

T	H	e	l	l	**o**		W	o	r	l	d
P				l	**o**						
pos	0	1	2	**3**	4	5	6	7	8	9	10

Если все символы образца совпали с символами текста, то есть индекс i становится равным m, значит образец содержится в тексте в позиции pos. Если же в результате сдвигов совмещаются последние символы образца и текста (pos = m – n) и, при этом, какой-то из символов не совпадает, сдвигать образец дальше не имеет смысла – образец в тексте на содержится.

T	+	+	+	+	+	+	+	+	+	+	+
P							*	*	*	*	*
pos	0	1	2	3	4	5	**6**	7	8	9	10

В данном примере n = 11, m = 5, поэтому pos = m – n = 6.

Количество сравнений двух символов в этом алгоритме в среднем пропорционально m * n.

3. Алгоритм Бойера-Мура-Хорспулла

Вариант, предложенный Хорспуллом представляет собой упрощение оригинального алгоритма Бойера-Мура. Метод создания алгоритма – пространственно-временной компромисс. Анализируется образец и

формируется дополнительная структура данных – таблица (table) правых вхождений символов. Таблица строится для всех символов алфавита, то есть ее размер совпадает с мощностью алфавита (|A|). Значение кода символа – целое число, используется как индекс (index) элемента таблицы, содержащего позицию последнего (самого правого) вхождения этого символа в образце. Таким образом таблица – это массив целых чисел. Если символ не содержится в образце, то в таблице ему соответствует значение -1. В таблицах ASCII, Unicode 16, Unicode 32 заглавная латинская буква A имеет код 65, B – код 66 и т.д. Тогда образцу

P	A	B	C	A	B	B
index	0	1	2	3	4	5
code	65	66	67	65	66	66

Соответствует таблица

table	-1	-1	**3**	**5**	**2**	-1	-1
index	…	64	**65**	**66**	**67**	68	…

Так же, как и в алгоритме линейного поиска "накладываем" образец на текст с некоторой позиции pos в тексте. Изначально pos = 0. Далее попарно сравниваются символы образца и текста, но не с начала, а с конца образца. Если символы не совпадают, образец сдвигается по тексту вправо в общем случае не на одну, а не несколько позиций сразу. Величина сдвига определяется в соответствии с таблицей правых вхождений символов (table). Чтобы определить эту величину, во-первых, получаем позицию несовпадения **в образце** j, во-вторых, код **символа текста** не совпавшего с символом образца T[pos + j] и позицию его последнего появления в образце table[T[pos + j]]. Вычисляем значение сдвига, shift = j - table[T[pos + j]].

Например,

T	C	A	A	B	**C**	B	A	B	C	A	B	B	A
P	A	B	C	A	B	B							
j	0	1	2	3	**4**	5							

pos = 0, j = 4, table[T[pos + j]] = table[67] = 2, shift = 4 – 2 = 2

Сдвигаем образец на 2 позиции вправо: pos = pos + 2. Сдвиг выполняется таким образом, чтобы символ 'C' в образце совпал с таким же символом в тексте.

T	C	A	A	B	C	B	**A**	B	C	A	B	B	A
P			A	B	C	A	B	B					
j			0	1	2	3	**4**	5					

Теперь pos = 2, j = 4, table[T[pos + j]] = table[65] = 3, shift = 4 – 3 = 1

Сдвигаем образец на 1 позицию вправо: pos = pos + 1.

T	C	A	A	B	C	B	A	B	**C**	A	B	B	A
P				A	B	C	A	B	B				
j				0	1	2	3	4	**5**				

pos = 3, j = 5, table[T[pos + j]] = table[67] = 2, shift = 5 – 2 = 3

Сдвигаем образец на 3 позиции вправо: pos = pos + 3.

T	C	A	A	B	C	B	A	B	C	A	B	B	A
P							A	B	C	A	B	B	
j							0	1	2	3	4	5	

pos = 6, все символы совпадают – результат работы алгоритма значение pos.

В процессе поиска могут возникать ситуации, подобные следующей

T	C	A	A	**B**	B	B	A	B	C	A	B	B	A
P	A	B	C	A	B	B							
j	0	1	2	**3**	4	5							

Здесь j = 3, table[66] = 5 и shift = 3 – 5 = -2. Сдвиг оказывается величиной отрицательной то есть предполагает сдвиг образца влево. Сдвигать образец влево не имеет смысла, поэтому такая ситуация для алгоритма является "неудобной". В варианте Хорспулла решение – это возврат к стратегии линейного поиска, то есть сдвиг вправо на один символ. Другими словами, если shift < 1 выполняем shift = 1.

Еще один особый случай возникает, если несовпадающего символа текста вообще нет в образце

T	C	A	A	**D**	B	B	A	B	C	A	B	B	A
P	A	B	C	A	B	B							
j	0	1	2	**3**	4	5							

В этом случае, очевидно, нужно сдвинуть образец за этот символ (нигде не может произойти совпадение с D).

T	C	A	A	**D**	B	B	A	B	C	A	B	B	A
P					A	B	C	A	B	B			
j					0	1	2	3	4	5			

Для алгоритма это самая выгодная ситуация – сдвиг оказывается максимальным. В данном примере величина сдвига 4, а j = 3. Возвращаясь к

формуле shift = j – table[68] видим, что table[68] должно быть равно -1. Вот почему в таблице правых вхождений для всех символов, не встречающихся в образце, мы записываем значение -1.

В среднем количество основных операций, выполняющихся при работе алгоритма пропорционально m + n.

4. Алгоритм Карпа-Рабина

Алгоритм Карпа-Рабина основан на методе преобразования. Вместо сравнения строк многократно сравниваем целые значения. Для этого используем функцию, вычисляющую хэш-значение (целое число) для строки из m символов. m – это размер образца.

1. Один раз вычисляем хэш-значение для образца hash(P) и запоминаем его, например, как hp.

2. Выбираем m подряд расположенных символов текста, начиная с некоторой позиции pos, S_{pos}. В начале pos = 0.

3. Вычисляем хэш-значение для S_{pos} и сохраняем, например, в ht, ht = hash(S_{pos}).

4. Если hp ≠ ht, точного совпадения нет, смещаем образец по тексту вправо на одну позицию, pos = pos + 1. Повторяем все, начиная с пункта 3.

5. Если hp = ht – возможно совпадение. Чтобы проверить так ли это, посимвольно сравниваем образец и S_{pos}. Нет точного совпадения – возвращаемся к пункту 3, есть совпадение – завершаем работу алгоритма и возвращаем значение pos.

6. Если прошли по всему тексту и не нашли совпадения возвращаем признак отсутствия образца в тексте.

Так как максимальное количество сдвигов образца n – m + 1, каждому сдвигу предшествует вычисление значения хэш-функции, плюс нужно еще вычислить хэш для образца, всего может потребоваться до n – m + 2 вызовов хэш-функции.

Очевидно, что эффективность алгоритма будет определяться эффективностью алгоритма вычисления хэш-значений. Если алгоритм, реализованный в хэш-функции будет иметь линейную сложность, общая оценка эффективности алгоритма Карпа-Рабина будет O(n * m), то есть не лучше оценки алгоритма линейного поиска. Нужен более эффективный алгоритм.

Рассмотрим некоторый простой алгоритм хэширования строк. Если мощность алфавита |A|, коды символов находятся в диапазоне [0..|A|-1]. Например, можно рассмотреть десятичную систему счисления. В ней 10 цифр, от 0 до 9 включительно. Это алфавит с мощностью 10. Используя этот алфавит, мы можем формировать целые числа – последовательности из произвольного количества десятичных цифр. Каждой последовательности десятичных цифр, например, 12345 можно сопоставить ее значение, рассчитываемое по формуле $1*10^4 + 2*10^3 + 3*10^2 + 4*10^1 + 5*10^0$ или, в общем виде

$$\text{hash(S)} = \sum_{i=0}^{|S|} |A|^{|S|-i-1} ord(S_i) \qquad (1)$$

Здесь S – последовательность (12345 в данном примере), |S| - длина последовательности (5 в данном примере), |A| - мощность алфавита (10 в данном примере), $ord(S_i)$ – код i-го символа последовательности S. В случае произвольных наборов допустимых символов такой подход к вычислению хэш-значения можно применять, если для представления каждого символа алфавита используется одинаковое число бит. Например, таблица ASCII состоит из 128 символов, расширенная таблица ASCII из 256, набор Unicode 16 – из 65536 символов. Если используется расширенная таблица ASCII (|A| =

256), то для текста "ABC" хэш-значение равно 256^2*65 + 256^1*66 + 256^0*67 = 4259840 + 16 896 + 67 = 4276803. Для длинных последовательностей символов (а в случае алфавитов большой мощности и для достаточно коротких) будут получаться большие значения. На практике нужны хэш-значения из ограниченного диапазона [0..k-1]. Их можно получить как остаток от деления рассчитанного большого числа на k. Например, если нужен диапазон [0..14280], то k = 14281 и получится хэш-значение 4276803 mod 14281 = 6 784. В итоге хэш-значение в алгоритме Карпа-Рабина можно вычислять следующим образом. Для образца и для первых m символов текста находим хэш-значение по приведенной выше формуле, то есть как значение полинома m-1 степени (например, по схеме Горнера). Для последовательности из следующих m символов текста можно использовать полученное прежде хэш-значение. Например, если находим хэш-значения для последовательностей из трех символов (m = 3) в тексте "ABCD". Для последовательности S_0 = "ABC" из первых трех символов получаем (до получения остатка от деления на k), как показано выше, hash(S_0) = 256^2*65 + 256^1*66 + 256^0*67. Для получения hash(S_1), где S_1 = "BCD", по той же формуле нужно было бы составить выражение 256^2*66 + 256^1*67 + 256^0*68, но видно, что можно, во-первых, удалить из hash(S_0) вклад первого символа S_0, то есть 256^2*65, получим 256^1*66 + 256^0*67, во вторых "сдвинуть" символы 'B' и 'C' так, чтобы последовательность S_1 начиналась с них, этого можно добиться умножая 256^1*66 + 256^0*67 на 256 (то есть на |A|), получим 256^2*66 + 256^1*67, наконец, в-третьих, остается добавить код символа 'D' (68) и мы получим ожидаемое выражение 256^2*66 + 256^1*67 + 256^0*68. В общем случае вычисление выполняются по формуле

$$hash(S_{i+1}) = |A|\left(hash(S_i) - |A|^{m-1} ord\left(S_{i,0}\right)\right) + ord(S_{i+1,m-1}), \qquad (2)$$

здесь S_i – предыдущая последовательность, $S_{i,0}$ – начальный символ этой последовательности, $S_{i+1,m-1}$ – последний символ новой последовательности. $|A|^{m-1}$ – всегда одно и то же, то есть вычисляется один раз. Видно, что независимо от S_i нужно выполнить два умножения, вычитание и сложение, то есть имеем алгоритм константной сложности.

Если хэш-значения образца и S_i совпали, необходимо сравнить образец с этой последовательностью посимвольно из-за возможных коллизий в значениях, выдаваемых хэш-функцией. Очевидно, что, если таких промахов много алгоритм становится неэффективным. Хорошая хэш-функция должна генерировать значения достаточно равномерно на заданном диапазоне [0..k-1]. В приведенном примере этого можно достичь, выбирая в качестве k достаточно большое простое число (например, 14281 – простое число), не слишком близкое к $2^i - 1$ [5].

Инструкция по технике безопасности при проведении работы

При выполнении работы нужно выполнять требования по технике безопасности при работе с электронными приборами.

Цель работы

Изучение алгоритмов поиска образца в тексте. Рассмотрение различных методов создания алгоритмов для решения одной и той же задачи.

Рабочее задание

ЗАДАНИЕ 1. Реализуйте алгоритм линейного поиска образца в тексте. Организуйте подсчет и вывод количества выполненных основных для алгоритма операций.

ЗАДАНИЕ 2. Сгенерируйте случайным образом различные по длине последовательности символов в качестве текста и образца и проверьте соответствует ли количество выполненных в Вашей реализации основных операций ожидаемой оценке эффективности алгоритма. Разделите случаи, когда образец встречается в тексте и когда его в тексте нет.

ЗАДАНИЕ 3. Реализуйте алгоритм Бойера-Мура-Хорспулла с подсчетом и выводом количества выполненных основных для алгоритма операций.

ЗАДАНИЕ 4. Протестируйте работу Вашей реализации алгоритма Бойера-Мура-Хорспулла с текстами и образцами из задания 2. Сравните результаты работы алгоритмов линейного поиска и Бойера-Мура-Хорспулла.

ЗАДАНИЕ 5. Так как Ваша реализация алгоритма Бойера-Мура-Хорспулла – это программа на конкретном языке программирования, для текста и образца Вы используете алфавит, заданный в этом языке программирования для тестовых строк. Измените реализацию алгоритма так, чтобы она могла работать с некоторым предопределенным алфавитом (который передается алгоритму в качестве входного параметра). Этот алфавит будет состоять из множества символов, являющихся подмножеством символов алфавита использованного Вами языка программирования.

ЗАДАНИЕ 6. Протестируйте реализацию из задания 5, используя какой-нибудь ограниченный алфавит, например, состоящий только из символов, представляющих арабские цифры.

ЗАДАНИЕ 7. Создайте функцию, реализующую вычисления по формуле (1) параграфа 4 "Алгоритм Карпа-Рабина". Входные параметры: последовательность символов S (длины не меньше m), количество символов m. Функция берет первые m символов S и возвращает вычисленное по формуле (1) большое целое значение (в Java, например, нужно использовать тип BigInteger). Используется алфавит заданный для строк в выбранном Вами языке программирования.

ЗАДАНИЕ 8. Создайте хэш-функцию, описанную в параграфе 4 "Алгоритм Карпа-Рабина". Входные параметры: последовательность символов S, количество символов m по которым вычисляется хэш-значение, позиция pos, начиная с которой берутся символы, число k, определяющее диапазон генерируемых хэш-значений, большое целое число value, по которому рассчитывался хэш для предыдущей последовательности.
Если pos = 0, текущее значение value игнорируется, вызывается функция из задания 7, возвращаемое ею значение записывается в value, вычисляется и возвращается результат остатка от его деления на k.

Если pos ≠ 0, то value содержит значение по предыдущей последовательности, необходимое в вычислениях по формуле (2) параграфа 4 "Алгоритм Карпа-Рабина". В value содержится значение Si из этой формулы. Большое целое значение, полученное по формуле (2), заносится в value, затем вычисляется и возвращается результат остатка от его деления на k.

ЗАДАНИЕ 9. Реализуйте алгоритм Карпа-Рабина, используя хэш-функцию из задания 8.

ЗАДАНИЕ 10. Протестируйте реализацию алгоритма Карпа-Рабина из задания 9, используя тексты и образцы разного размера.

Общие положения

Поиск образца в тексте – это важная операция в задачах обработки текстовой информации. Существуют алгоритмы как точного, так и приближенного поиска. В настоящей работе рассмотрены некоторые алгоритмы точного поиска образца в тексте, построенные с использованием разных принципов (методов) создания алгоритмов.

Материально-техническое обеспечение работы

Компьютеры с установленным программным обеспечением:

1. ОС Windows или Linux.
2. Java SDK 11+ или Python 3.
3. IDE для работы с Java- или Python-проектами.

Порядок выполнения работы

1. Ознакомиться с теоретическим материалом.
2. Выполнять задания в порядке их следования так как при выполнении текущего задания потребуются результаты, полученные при выполнении предыдущих заданий.

Контрольные вопросы

1. Какая операция является основной при поиске образца в тексте?
2. Что является результатом поиска образца в тексте?
3. Какова оценка эффективности алгоритма линейного поиска образца в тексте?

4. Какие алфавиты для представления текста Вы знаете? Чем они отличаются?

5. Какой метод создания алгоритмов применен при построении алгоритма линейного поиска образца в тексте?

6. Какова оценка эффективности алгоритма Бойера-Мура-Хорспулла?

7. Какой метод создания алгоритмов применен при построении алгоритма Бойера-Мура-Хорспулла?

8. Что представляет собой таблица правых вхождений в алгоритме Бойера-Мура-Хорспулла?

9. Каков размер таблицы правых вхождений в алгоритме Бойера-Мура-Хорспулла?

10. Какое значение в таблице правых вхождений алгоритма Бойера-Мура-Хорспулла соответствует символу, не встречающемуся в образце?

11. Ситуация, когда в тексте встречается символ, отсутствующий в образце, является выгодной или нет для алгоритма Бойера-Мура-Хорспулла?

12. Какой особый случай необходимо рассматривать в алгоритме Бойера-Мура-Хорспулла?

13. Как обрабатывается особый случай в алгоритме Бойера-Мура-Хорспулла?

14. Какова оценка эффективности алгоритма Карпа-Рабина?

15. Какой метод создания алгоритмов применен при построении алгоритма Карпа-Рабина?

16. Для чего в методе Карпа-Рабина используется хэш-функция?

17. Найдено ли вхождение образца в тексте в позиции pos, если хэш-значения образца и фрагмента теста такого же размера, начинающегося с позиции pos совпадают?

18. Как эффективность метода Карпа-Рабина зависит от эффективности алгоритма хэширования?

19. Будет ли эффективной реализация метода Карпа-Рабина если оценка эффективности алгоритма хэширования линейная?

20. Можно ли с помощью методов линейного поиска образца в тексте, Бойера-Мура-Хорспулла, Карпа-Рабина найти все вхождения образца в тексте?

Перечень использованных информационных ресурсов

1. П.Луридас "Алгоритмы для начинающих: теория и практика для разработчика". – М: Эксмо. – 2018, 608 с.
2. А.Левитин "Алгоритмы: введение в разработку и анализ". – М: ООО "И.Д. Вильямс". – 2018, 576 с.
3. Н.Вирт "Алгоритмы и структуры данных". – М: ДМК Пресс. – 2016, 272с.
4. Т.Х.Кормен, Ч.И.Лейзерсон, Р.Л.Ривест, К.Штайн "Алгоритмы: построение и анализ", 3-е изд. – СПб.: ООО "Диалектика". – 2019, 1328 с.
5. С.Скиена "Алгоритмы: руководство по разработке", 2-е изд. – СПб: БХВ-Петербург. – 2016, 720с.

Редактор А.А. Литвинова

ЛР № 04779 от 18.05.01.	В набор	В печать
Объем 0,5 усл.п.л., уч.-изд.л.	Офсет.	Формат 60x84/16.
Бумага тип №3.	Заказ №	Тираж 120. Цена

Издательский центр ДГТУ

Адрес университета и полиграфического предприятия:

344010, г. Ростов-на-Дону, пл. Гагарина, 1.